Project Proposal

Chase Rush

A Top-Down Police Chase Survival Game

Programming 2 | Kasetsart University (CPE)

1. Project Overview

Chase Rush is a top-down 2D survival game developed using Python and Pygame. The player controls a Lamborghini navigating an infinite desert world while being chased by AI police vehicles. The game ends when the player's HP reaches zero from collisions. Difficulty increases over time as more police units are deployed and their speed scales up. The project includes both a fully playable game module and a multi-section statistics dashboard that collects, processes, and visualizes gameplay data per frame and per run.

The final implementation significantly expanded beyond the original proposal, adding physics-based movement, a persistent wallet system, gift box mechanics, banana and cactus obstacles, smoke/tire effects, and a 6-section data dashboard covering EDA, distribution analysis, correlation, statistical summaries, and time-series charts.

2. Project Review

Reference Project: Smashy Road: Wanted 2 (Bearbit Studios, 2017)

Smashy Road: Wanted 2 is a top-down car chase game where players drive various vehicles while being pursued by police. The core loop involves surviving as long as possible while destroying police vehicles and collecting power-ups. The game features procedurally generated environments and a large roster of unlockable vehicles.

Key improvements in Chase Rush:

- Statistics & Analytics: Chase Rush collects detailed per-frame gameplay statistics and provides a multi-section data visualization dashboard — covering EDA, distribution analysis, correlation, time-series charts, and statistical summaries.
- Designed for Desktop: Optimized for keyboard-based controls on PC using Pygame, unlike the mobile-first design of Smashy Road.
- Transparent Difficulty Scaling: Police spawn rates and speed increase at defined stage intervals that the player can observe through the dashboard.
- Persistent Economy: A wallet system saves money across sessions, adding long-term progression absent from Smashy Road.

- Data-Driven Design: Gameplay parameters can be tuned based on collected statistical data; IQR outlier detection reveals exceptional and poor runs automatically.

3. Programming Development

3.1 Game Concept

The player drives a Lamborghini on an infinite top-down desert map. Police cars spawn at the edges and chase the player using physics-aware pursuit AI. The game ends when HP drops to zero from police collisions. Score is based on survival time, money earned, and stage reached. Difficulty scales via a stage system that increases police count and speed over time. The player can collect gifts for bonuses, pick up banknotes for money, and use nitro to outrun police.

Key Features (Final Implementation):

- Survival-based gameplay with HP system and progressive difficulty stages
- Physics-based driving: acceleration, friction, lateral grip, drag
- AI police cars with dynamic difficulty scaling (apply_difficulty())
- Gift box system: money, nitro refill, invincibility, ram boost
- Banana peels (loss of control) and cactus obstacles (collision damage)
- Persistent wallet saved to CSV across sessions
- Per-frame stat logging to CSV + auto-generated dashboard PNG after each run
- Tire skid marks, smoke/fire particle effects, engine sounds

3.2 Object-Oriented Programming Implementation

The following 12 classes are implemented in the final project:

Class	Key Attributes / Methods	Role
Game	handle_events(), update(), render(), _spawn_police(), _kill_police()	Central controller. Manages game loop, collision resolution, spawning, and rendering. Owns all subsystems.
Player	x, y, hp, nitro_level move(), coast(), draw(), get_hit_poly()	Physics-based player car. Handles keyboard input, nitro management, HP tracking, and polygon hitbox.
Police	x, y, hp, stun_frames update(), stun(), apply_difficulty()	AI-controlled police car. Seeks player using physics-aware pursuit. Supports stun and dynamic difficulty scaling.
Map	obstacles, width, height generate(), update(), pop_gift_hit(), pop_banana_hit()	Generates and manages the infinite world: roads, cacti, banknotes, bananas, and gift boxes.
Wallet	balance, total_earned add(), spend()	Persistent money balance saved to CSV. Flushes every change immediately to prevent data loss.
StatsTracker	session_data, gift_events log_event(), log_gift(), save_csv(), append_run_summary()	Logs per-frame events and per-run summaries to CSV files for dashboard analysis.
Dashboard	data_frame, runs_frame load_data(), plot_charts()	Loads CSV data and renders a 6-section PNG visualization dashboard using Matplotlib and Pandas.
SoundManager	channels, sounds play_loop(), play_once(), start_game_ambient()	Manages all game audio: background music, engine sounds, crash and nitro SFX.
TireMarkManager	segments add_wheel_pair(), break_stroke(), draw()	Renders connected skid mark trails from both rear wheels of the player car.

DripSmokeFX	_parts burst(), update(), draw()	Short-lived smoke/dust particle puffs on hard slides. Color adapts to road surface.
EngineSmokeFX	_parts emit(), emit_nitro(), explode(), update()	Hood smoke or fire particles from damaged vehicles. Switches to fire palette at low HP.
_NoInput	__getitem__()	Internal sentinel. Returns False for any key query — safe fallback before input is available.

3.3 Algorithms Involved

- Chase AI (Physics-Aware Pursuer): Each Police object calculates the angle toward the player each frame and applies the same friction/drag physics as the player to produce realistic pursuit behavior rather than instant-turn homing.
- Collision Detection: Polygon-based collision (Separating Axis Theorem via polys_intersect()) is used for player-police interactions. Rectangle-based collision handles player-obstacle and pickup detection.
- Difficulty Scaling: apply_difficulty() scales police top speed and acceleration based on elapsed time, creating observable stage transitions that the player and dashboard can track.
- Data Logging Pipeline: StatsTracker records one row per frame (60 fps) to gameplay_stats.csv, and one row per run to game_runs.csv, using Python's csv module. The Dashboard class loads both with Pandas for analysis.
- IQR Outlier Detection: The stats table identifies outliers per column using Q1 - 1.5*IQR and Q3 + 1.5*IQR thresholds, flagging exceptional or anomalous runs automatically.

4. Statistical Data (Prop Stats)

4.1 Data Features

The following features are tracked every frame (60 fps) during each gameplay session via StatsTracker.log_event(), accumulating thousands of records per run:

Feature	Why valuable?	How to obtain 100+ values?	Class / Source	Visualization
Player Speed	Reveals movement patterns and pressure response	Logged every frame — 100+ records in < 2 seconds	StatsTracker (player_speed)	Line chart + trend line, Histogram
Distance to Nearest Police	Measures danger level over time	Logged every frame	StatsTracker (dist_nearest_police)	Line chart, Histogram
Player HP	Tracks survivability and damage events	Logged every frame	StatsTracker (player_hp)	Line chart
Wallet Balance	Tracks economy progression during the run	Logged every frame	StatsTracker (wallet_balance)	Area chart with gift markers
Police Stage	Shows difficulty scaling over time	Logged every frame	StatsTracker (police_stage)	Stage shading on charts
Total Cactus / Banana / Collision Hits	Quantifies risk-taking behaviour	Logged every frame (cumulative)	StatsTracker (cumulative counters)	Stats table, Bar chart
Gifts Collected (by type)	Reveals reward strategy per stage	One row per gift pickup	StatsTracker.log_gift() gift_events.csv	Pie chart, Bar chart

4.2 Data Recording Method

Per-frame gameplay data is saved to gameplay_stats.csv using Python's csv module via StatsTracker.save_csv(). One record is appended per frame (60 fps) throughout the session, yielding thousands of records per run. Per-run summary data is appended to game_runs.csv by append_run_summary() at game over. Gift pickup events are separately logged to gift_events.csv. All three files are loaded by the Dashboard class using Pandas (pd.read_csv()) for cleaning, analysis, and visualization.

4.3 Data Analysis Report

The Dashboard class loads the CSV files using Pandas and generates a 6-section PNG visualization using Matplotlib:

Graph	Feature	Objective	Type	Axes
1	Duration / Top Speed / Money	Show distribution shape and skewness	Histogram	Value → Frequency
2	Multiple numeric columns	Compare spread and outliers across metrics	Boxplot	Column → Value
3	Duration vs Money / Speed vs Kills	Show correlation between variables (ρ)	Scatter plot	Variable A → Variable B
4	All key metrics (runs)	Descriptive statistics + IQR outliers	Stats table	Metric → Mean/Med/Mode/Max/Min/Std/Outliers
5	Player speed per frame	Speed progression + trend over run	Line + trend line	Time (s) → Speed
6	Police distance per frame	Danger level over time	Line chart	Time (s) → Distance
7	Wallet balance per frame	Economy story of the run	Area chart	Time (s) → Balance
8	Gift type counts	Prize distribution proportion	Pie chart	Category → Count

Statistical Values Table (actual data from 20 recorded runs):

Feature	Mean	Median	Mode	Max	Min	Std Dev
Player Speed (km/h)	27.2	28.3	29.8	39.0	0.0	9.4
Dist. Nearest Police (px)	312	285	220	980	12	198
Top Speed (km/h)	34.1	33.5	31.2	39.0	24.8	3.6
Duration (s)	48.3	38.5	22.1	182.0	8.2	38.7
Money Earned	156	120	80	520	20	118
Police Killed	1.35	1.0	0	6	0	1.42
Banana Slips	0.95	0.5	0	5	0	1.28
Cactus Hits	1.20	1.0	0	4	0	1.06

5. Project Planning Timeline

Week	Course Week	Task

1	8	Proposal submission / Project setup, GitHub repo initialization
2	9	Implemented Player (physics movement), Map (world generation, obstacles), basic Pygame rendering
3	10	Implemented Police class with physics-aware chase AI, polygon collision detection (SAT)
4	11	Implemented StatsTracker (per-frame logging), Wallet (persistent CSV), difficulty scaling system
5	12	Implemented Dashboard (6-section visualization), gift/banana/cactus mechanics, particle effects
6	13	Testing, bug fixing, dashboard refinement, data collection (20+ runs, thousands of records)
7	14	Final documentation: DESCRIPTION.md, README.md, screenshots, VISUALIZATION.md, GitHub tag

6. Document Version

Version	2.0 (Final)
Date	10 May 2026
Editor	Kawin Kaewparadai 6710545431

Version history and TA feedback:

Date	Name	Feedback / Action Taken
16/03	Amornrit	Requested more feature depth, obstacle details, and recommended per-frame data collection over session-level metrics. → Resolved: switched to per-frame logging (60 fps) and added gift/banana/cactus mechanics.
17/03	Peraya	Requested inheritance/polymorphism in class design and correct per-frame data strategy. → Resolved: expanded to 12 classes with clear relationships; StatsTracker logs per-frame data accumulating thousands of records per run.
04/04	Peraya	Missing statistical values table; wrong graph type for Player Position. → Resolved: added full stats table (mean, median, mode, max, min, std dev, outliers) to dashboard Section 05; replaced bar graph with scatter plot (heatmap) for position analysis.
10/05	—	Version 2.0: Final submission. All TA feedback addressed. Dashboard expanded to 6 sections. All required documentation complete.